Texas A&M University

Department of Nuclear Engineering

NUEN 430

Final Project Report

Minecraft Nuclear Reactor

Transient Two-Group Neutron Diffusion Solver for a Modular Block-Based Minecraft Core

Prepared by

Tyler Scott

UIN 433009907

Submitted to

Dr. Jean Ragusa

30 October 2026

Contents

I Proposal **3**

1 Title and team **3**

2 Problem Statement | Geometry **3**
2.1 Boundary 5
2.2 Equations 5
2.2.1 Spatial 5

3 Numerical Methods **6**
3.0.1 Transient 7

4 Verification Plan **9**
4.1 Transient 9

5 Deliverables **9**

List of Tables

1 Two-group nuclear data for the reactor regions [3]. 4

List of Figures

1 Proposed core layout inspired by the 3D IAEA PWR benchmark presented by Soliman [3]. Twelve fuel blocks (orange) are surrounded by twelve reflector blocks (blue). Control rod positions are centered at four-block intersections, with inserted regions represented by combined fuel-plus-control-rod cross sections. 4

Introduction

Minecraft blocks are uniquely suited for building modular reactor cores because of their simple preordained grid spacing and metric unit system. Blocks are 1 meter cubes and can easily be made to represent fuel assemblies and can thus be constructed into a simulatable core by querying face boundaries of each core block.

Inspiration

A mod properly implementing core behavior can be made into an add-on mod for the very popular Create mod [1] in order to generate large quantities of stress units (SU), the mod's power system, for use in automating tasks and powering various machines.

Scope

This project mainly focuses on solving a time-dependent reactor with and without a face neutron source using discretization and integration methods developed in class. Particles will not be ray-traced to simulate radiation physics as they are in the CROWNS mod [2], as this project mainly aims to provide means of power and an accurate operational experience of a critical reactor in Minecraft.

This project will not make use of solutions to methods of heat transfer or removal from the core. The Create CROWNS mod [2] is a nuclear science mod that allows the player to construct and make energy from nuclear reactors operating on uranium, plutonium, and thorium fuels. CROWNS contains infrastructure to make use of power produced by the core and has means to turn it into steam and thus rotational power via other machines present in the mod library, and power extraction from the core is thus replaced with a black box system and constant coolant flow metrics.

Part I

Proposal

1 Title and team

Title. Transient Two-Group Neutron Diffusion Solver for a Modular Block-Based Minecraft Core

Team. Tyler Scott

2 Problem Statement | Geometry

The core is a 3D Cartesian arrangement of 1-meter cubic blocks, subdivided as necessary for the solver. The core layout is inspired by the 3D IAEA PWR benchmark geometry presented by Soliman [3], adapted to the modular block-based construction of Minecraft. Candidate blocks consist of fuel assemblies and reflectors with homogenized two-group cross sections. Control rod locations are centered at the intersections of four adjacent fuel blocks. Rather than modeling the rod and fuel separately, the affected regions are represented using the combined fuel-plus-control-rod cross sections (Region 3) from the benchmark. This is an easy rule to follow via face querying, and the only necessary information to determine cross sections in the core is then the current height of each control rod. The standard layout is shown in Figure 1.

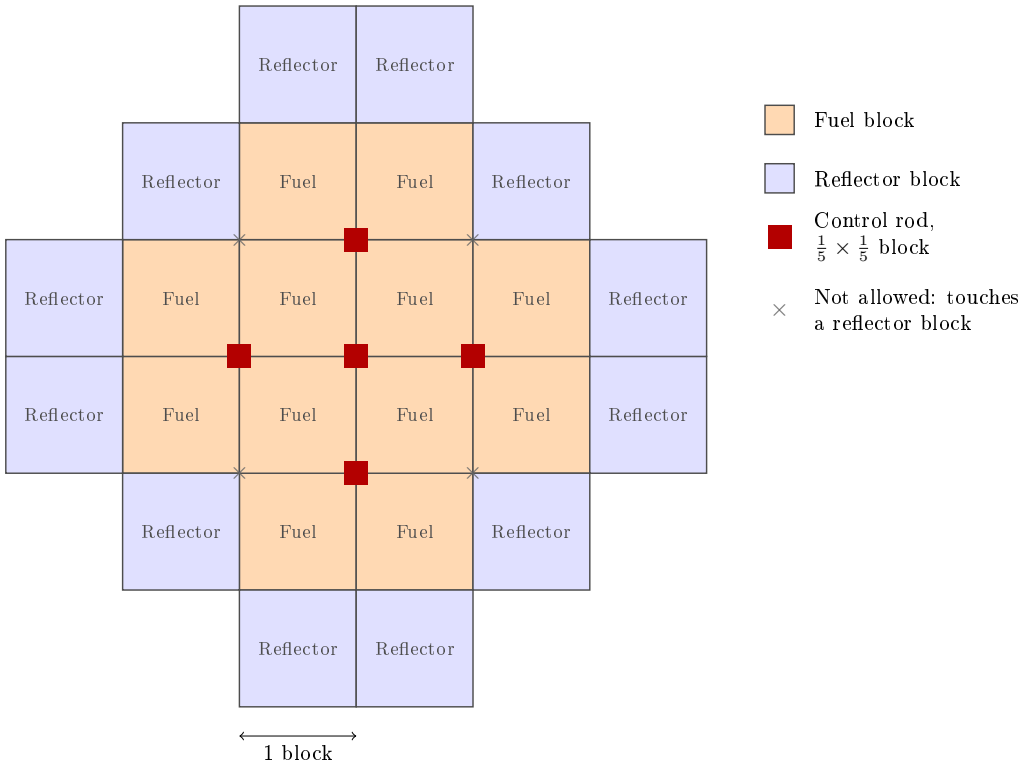


Figure 1: Proposed core layout inspired by the 3D IAEA PWR benchmark presented by Soliman [3]. Twelve fuel blocks (orange) are surrounded by twelve reflector blocks (blue). Control rod positions are centered at four-block intersections, with inserted regions represented by combined fuel-plus-control-rod cross sections.

Minecraft blocks must be meshed via at least 5 subdivisions per block in x and y to accomodate the 20 cm mesh sizing of the IAEA PWR benchmark.

The homogenized two-group constants are taken from the 3D IAEA PWR benchmark [3] using cross sections that resemble PWR UO2 fuel. The fast and thermal groups are denoted by subscripts 0 and 1, respectively. All macroscopic cross sections are in cm^{-1}, and diffusion coefficients are in cm.

Table 1: Two-group nuclear data for the reactor regions [3].

Region	Material	D_1	D_2	$\Sigma_{1\to2}$	Σ_{a1}	Σ_{a2}	$\nu\Sigma_{f2}$
1	Fuel 1	1.5	0.4	0.02	0.01	0.080	0.135
2	Fuel 2	1.5	0.4	0.02	0.01	0.085	0.135
3	Fuel 2 + Rod	1.5	0.4	0.02	0.01	0.130	0.135
4	Reflector	2.0	0.3	0.04	0.00	0.010	0.000
5	Reflector + Rod	3.0	0.3	0.04	0.00	0.055	0.000

Reflector + Rod cross sections would be unused for the configuration shown in Figure 1, but would be used if rods are made use of at teh fuel-reflector boundaries. The layout would be read from Minecraft or would be imposed when testing the solver, at which point control rod slots would be

calculated. Because of the nature of the blocks and symmetry of the core, the grid will be equivalent in x and y and may be simplified in z to conserve compute.

2.1 Boundary

Vacuum boundary conditions are imposed at the outer surfaces of the reflector regions, representing the absence of incoming neutron current. Following the extrapolated vacuum boundary approximation used by Soliman [3], the condition for each energy group is

$$\phi_g + 2.1312 D_g \frac{\partial \phi_g}{\partial n} = 0$$

where n is the outward surface normal and D_g is the diffusion coefficient of the boundary material. At interfaces between fuel and reflector regions, neutron flux and normal current are continuous

$$\phi_g^- = \phi_g^+, \qquad D_g^- \frac{\partial \phi_g^-}{\partial n} = D_g^+ \frac{\partial \phi_g^+}{\partial n}.$$

The outer boundary generally lies one meter beyond the fuel regions, although some corner locations have shorter reflector paths and consequently greater leakage. Additional reflector blocks may be added to complete the square computational domain if deemed necessary.

2.2 Equations

The core can be solved in a few ways, one being solving the shape of the core once and assuming it remains that way upon adding reactivity with rods, and magnifying the power based on the time evolution component. This is probably satisfactory radially if the rods are constrained to move in gang but inaccurate along the core axis as the rods are extracted. Another method is modeling changes in core cross sections as rods are extracted, but this requires rebuilding the A and F matrices every time reactivity changes.

An ideal mix of the two is to simulate multiple steady state shapes of the core at various rod configurations, and to interpolate the matrices as a function of rod heights. This helps to preserve the shape without requiring constant recalculation of the core.

2.2.1 Spatial

The group g neutron density balance equation is

$$-\nabla \cdot (D_g \nabla \phi_g) + \Sigma_g^r \phi_g = \sum_{h \neq g}^{G} \Sigma_{h \to g} \phi_h + \frac{\chi_g}{k} \sum_{0}^{G} \nu_h \Sigma_h^f \phi_h$$

Rewritten

$$-\nabla \cdot (D_g \nabla \phi_g) + \Sigma_g^r \phi_g - \sum_{h \neq g}^{G} \Sigma_{h \to g} \phi_h = \frac{\chi_g}{k} \sum_{0}^{G} \nu_h \Sigma_h^f \phi_h$$

The finite volume method follows from integrating the balance equation in space. Specifically, volumetric integration forces a physical quantity conservation instead of density in space:

$$\int -\nabla \cdot (D_g \nabla \phi_g) dV + \Sigma_g^r \phi_g V_i - V_i \sum_{h \neq g}^{G} \Sigma_{h \to g} \phi_h = \frac{\chi_g}{k} V_i \sum_{0}^{G} \nu_h \Sigma_h^f \phi_h$$

Using Gauss' divergence theorem the volumetric leakage can be converted to a face normal leakage quantity representing the equivalent source or sink of flux from a volume of interest.

$$\int -\nabla \cdot (D_g \nabla \phi_g) dV \rightarrow \int_{\partial} -D_g \nabla \phi_g \cdot \vec{n} dA$$

3 Numerical Methods

The steady spatial problem is solved via multigroup **3D cell-centered FVM** running a **backward Euler transient** stepper. The integral can be decomposed into the sum of the six cube face leakages and replaced in the bal. eq. for a given group g

$$\boxed{\sum_{f}^{\in\{NESWTB\}} J_{f,g} \cdot \vec{n} A_f + \Sigma_g^r \phi_g V_i - V_i \sum_{h \neq g}^{G} \Sigma_{h \rightarrow g} \phi_h = \frac{\chi_g}{k} V_i \sum_{0}^{G} \nu_h \Sigma_h^f \phi_h}$$

J_f on all the respective positive and negative-side faces for the cell-centered scheme is

$$J_g^+ = -\bar{D}_g^+ (\phi_g^+ - \phi_g^i) \qquad J_g^- = -\bar{D}_g^- (\phi_g^i - \phi_g^-)$$

$$d_{f,g}^{\pm} = \left(\frac{h_i}{2D_g^i} + \frac{h_{\pm}}{2D_g^{\pm}} \right)^{-1} = \frac{2D_g^i D_g^{\pm}}{h_i D_g^{\pm} + h_{\pm} D_g^i}$$

Using G major vectors:

$$\begin{bmatrix} B_0 & -S_{01} & \cdots & -S_{0G} \\ -S_{10} & B_1 & \cdots & -S_{1G} \\ \vdots & \vdots & \ddots & \vdots \\ -S_{G0} & -S_{G1} & \cdots & B_G \end{bmatrix} \begin{bmatrix} \phi_0 \\ \phi_1 \\ \vdots \\ \phi_G \end{bmatrix} = \frac{1}{k} \begin{bmatrix} F_{00} & F_{01} & \cdots & F_{0G} \\ F_{10} & F_{11} & \cdots & F_{1G} \\ \vdots & \vdots & \ddots & \vdots \\ F_{G0} & F_{G1} & \cdots & F_{GG} \end{bmatrix} \begin{bmatrix} \phi_0 \\ \phi_1 \\ \vdots \\ \phi_G \end{bmatrix}$$

Where

$$(B_g)_{ii} = \sum_{j \in \mathcal{N}(i)} d_{ij,g} A_{ij} + \Sigma_{g,i}^r V_i \qquad (B_g)_{ij} = -d_{ij,g} A_{ij}, \quad j \in N(i)$$

$$S_{gh} = \text{diag}_{i=1}^{N} \left(V_i \Sigma_{h \rightarrow g,i} \right), \qquad h \neq g$$

$$F_{gh} = \text{diag}_{i=1}^{N} \left(V_i \chi_{g,i} \nu_h \Sigma_{h,i}^f \right)$$

The problem can now be written

$$\mathbf{L} = \mathbf{B} - \mathbf{S}$$

$$\mathbf{L}\boldsymbol{\Phi} = \frac{1}{k} \mathbf{F}\boldsymbol{\Phi}$$

$$\mathbf{L}^{-1}\mathbf{F}\boldsymbol{\Phi} = k\boldsymbol{\Phi}$$

This is not going to be used in practice, but eigenvalue iteration will be performed for verification of the spatial code. A transient code will be used in practice to run the reactor, so the eigenvalue iteration will not be consistently performed.

3.0.1 Transient

The spatial steady state equation is not very useful when you need to do anything with the reactor, so instead of equating the losses and gains we can find the change in flux in time via their difference in contributions.

$$V\frac{\partial n}{\partial t} = V\left[gain - loss\right]$$

For a given group g in cell i,

$$V_i\frac{\partial n}{\partial t} = V_i\frac{1}{v_g}\frac{\partial \phi_{g,i}}{\partial t} = -\left(\sum_{f\in\{N,E,S,W,T,B\}} J^{\text{out}}_{f,g}A_f + \Sigma^r_{g,i}\phi_{g,i}V_i\right) + V_i\sum_{h\neq g}^{G}\Sigma_{h\to g,i}\phi_{h,i} + V_i\chi_g\sum_{h=0}^{G}\nu_h\Sigma^f_{h,i}\phi_{h,i}$$

k is gone as it represented a rebalancing factor relating production to loss. Here the neutron change rate is solved for directly. It is important to distinguish prompt and delayed neutrons in temporal analysis; reactivity has to be found in relation to β, which requires precursor group analysis. A source vector S can also be added in the highest energy group at a face of interest at reactor startup.

Thus the real balance equation to solve becomes

$$\begin{aligned} V_i\frac{1}{v_g}\frac{\partial \phi_{g,i}}{\partial t} = & -\left(\sum_{f\in\{N,E,S,W,T,B\}} J^{\text{out}}_{f,g}A_f + \Sigma^r_{g,i}\phi_{g,i}V_i\right) \\ & + V_i\sum_{h\neq g}^{G}\Sigma_{h\to g,i}\phi_{h,i} \\ & + V_i\chi_g\left(\underbrace{(1-\beta)\sum_{h=0}^{G}\nu_h\Sigma^f_{h,i}\phi_{h,i}}_{\text{Prompt}} + \underbrace{\sum_{j=1}^{6}\lambda_j C_{j,i}}_{\text{Delayed}}\right) + S_{g,i}(t) \end{aligned}$$

Each of six delayed groups have their own balance

$$\frac{\partial C_{j,i}}{\partial t} = \underbrace{\beta_j\sum_{g=0}^{G}\nu_g\Sigma^f_{g,i}\phi_{g,i}}_{\text{Production}} - \underbrace{\lambda_j C_{j,i}}_{\text{Decay}}$$

We want to solve something akin to $\mathbf{y}' = \mathbf{A}\mathbf{y} + \mathbf{b}$ where we have a general system of ODEs and a source. We can turn the left hand side into the following:

$$V_i\frac{1}{v_g}\frac{\partial \phi_{g,i}}{\partial t} = \mathbf{A}\mathbf{\Phi}'$$

Since the IAEA benchmark is two-group,

$$\mathbf{A} = \begin{bmatrix}\mathbf{A_0} & 0\\ 0 & \mathbf{A_1}\end{bmatrix} \qquad \mathbf{A_g} = diag_i^N(\frac{V_i}{v_{g,i}})$$

This could also be written as a dot product of vectors but is less useful in the derivation. Also note that $\mathbf{\Phi} = \begin{bmatrix} \mathbf{\Phi_0} \\ \mathbf{\Phi_1} \end{bmatrix}$ where both $\mathbf{\Phi_g}$ are vectors of length n cells. Now we can reuse the matrices from the steady spatial problem

$$\mathbf{A\Phi'} = -\mathbf{L\Phi} + (\mathbf{1} - \beta)\mathbf{F\Phi} + \mathbf{DC} + \mathbf{S}$$

$\mathbf{D}$ is the delayed group coefficient matrix adding neutrons from delayed sources

$$\mathbf{D} \in \mathbb{R}^{G \times J} = \mathbb{R}^{2 \times 6} \qquad \mathbf{D_{gj}} = diag(V_i \chi_g \lambda_j)$$

We can build the precursor system as follows:

$$\mathbf{I}\frac{d\mathbf{C}}{dt} = \mathbf{P\Phi} - \mathbf{\Lambda C}$$

$$\mathbf{P_{gj}} \in \mathbb{R}^{J \times G}$$

Where $\mathbf{\Lambda}$ is the identity filled with the decay constants of each precursor group, and $\mathbf{P}$ is filled with the precursor production from delayed fission groups. We combine $\mathbf{\Phi}$and $\mathbf{C}$ into $\mathbf{y}$.

$$\mathbf{y} = \begin{bmatrix} \mathbf{\Phi} \\ \mathbf{C} \end{bmatrix}$$

Then the whole system can be written as

$$\underbrace{\begin{bmatrix} \mathbf{A} & 0 \\ 0 & \mathbf{I_6}(N) \end{bmatrix}}_{G} \mathbf{y'} = \underbrace{\begin{bmatrix} (1-\beta)\mathbf{F} - \mathbf{L} & \mathbf{D} \\ \mathbf{P} & -\mathbf{\Lambda} \end{bmatrix}}_{H} \mathbf{y} + \begin{bmatrix} \mathbf{S} \\ 0 \end{bmatrix}$$

$$\mathbf{Gy'} = \mathbf{Hy} + \mathbf{S}$$

$$\mathbf{y'} = (\mathbf{G^{-1}H})\mathbf{y} + \mathbf{G^{-1}S} \quad \rightarrow \quad \mathbf{G^{-1}H} = \mathbf{M}, \quad \mathbf{G^{-1}S} = \mathbf{b}$$

$$\mathbf{y'} = \mathbf{My} + \mathbf{b}$$

Now we can perform backward Euler time stepping. The time derivative of $\mathbf{y}$ gets approximated as the following

$$\frac{\mathbf{y^{k+1}} - \mathbf{y^k}}{\Delta t} = \mathbf{My^{k+1}} + \mathbf{b^{k+1}}$$

$$\boxed{[\mathbf{I} - \Delta t \mathbf{M^{k+1}}]\mathbf{y^{k+1}} = \mathbf{y^k} + \mathbf{\Delta t b^{k+1}}}$$

This is the resulting equation that is to be solved every tick in Minecraft. The game ticks (updates) 20 times a second, so this solution needs only to update the new $\mathbf{F}$ and $\mathbf{L}$ cross section matrices in order to excite a transient state, and the state vector and $\mathbf{b}$ are both always known starting from initial states. $\mathbf{y^k}$ is known from the beginning with zero flux and zero precursor concentration, and increase from source is what begins the process. If at any point the source is removed the solution simplifies. Backward Euler is always A- and L-stable, so the

A 20Hz game tick does not mean the solution is strictly solved 20 times per second. Reducing the solve time interval generally improves accuracy of the solution; the most updated output power is sent back to the game every 50 ms.

4 Verification Plan

The spatial problem can be verified by incrementally stepping up the problem complexity, starting from a single homogeneous fuel block to a gridded block, then perhaps a 2x2 core with perhaps one of the four blocks being modified from fuel to rod-fuel and verifying that k decreases with insertion. For a homogeneous cube with an extrapolated side length $\bar{a}$,

$$\phi_g(x, y, z) = \phi_{g,0} \cos\left(\frac{\pi x}{\bar{a}}\right) \cos\left(\frac{\pi y}{\bar{a}}\right) \cos\left(\frac{\pi z}{\bar{a}}\right)$$

the flux decreases in each direction as a cosine centered on the middle of the cube [5]. The extrapolated length can be found from the vacuum boundary condition

$$\phi_g + 2.1312 D_g \frac{\partial \phi_g}{\partial n} = 0$$

by adding an extrapolation distance $d_g = 2.1312 D_g$ to a [5].

$$\bar{a}_g = a + 2d_g = a + 4.2624 D_g$$

This verifies the shape of a homogeneous cube matches that of the analytical solution. Any number of subdivisions should match this case, with finer refinements appearing closest to the analytical solution. When performed in the full three dimensions, the proper fvm solution should show second-order with respect to uniform cell width h and 2/3-order with respect to the total number of cells in the simulation.

Face coupling is also something that needs to be verified in the spatial problem. This can be done by performing a 1D or 2D solution with two adjacent blocks in two scenarios: one where both blocks are fuel and the other where one of the blocks is a rod-fuel block. It should be visible that flux dips in the rodded region when compared to the unrodded solution.

4.1 Transient

The transient problem is verified in comparison to the eigenvalue-decomposition method of solving systems of ODEs where the final analytical solution becomes the following:

$$\mathbf{y} = \mathbf{V} \mathbf{e}^{\mathbf{\Lambda}(\mathbf{t}-\mathbf{t_0})} \mathbf{V}^{-\mathbf{1}} \mathbf{y_0}$$

in which the eigenvector matrix $\mathbf{V}$ and the eigenvalue identity Λ are the eigenpairs of $\mathbf{M}$ from the earlier derivation. This analytical solution verifies the no-source version of the problem and is not useful in verifying the source-based scenario. Since backward Euler is a first-order solution, it is expected that the error converges at a first-order slope in relation to the time step $\Delta t = t - t_0$.

5 Deliverables

The final project will consist of a functional transient two-group neutron diffusion solver for 3-dimensional block based user-defined geometries. It will make use of the standard six-group precursors and will use incremental control-rod movements to redefine cross sections in the core in order to make use of the backward Euler time step solver.

The following deliverables will be presented:

1. 3D fast and thermal flux distributions for select reactor configurations. Visualization of the change in flux when a rod is inserted at various heights.
2. The associated multiplication factors k for various steady state operations when rods are at aforementioned various heights.
3. Total reactor power as a function of time during "startup" from an initial power state (assuming a very low power initial criticality), during control rod insertion, and removal.
4. **Spatial convergence.** Tables/Plots verifying the expected second-order convergence of CC FVM. Plots verifying 2/3 order of convergence against total cell count in 3D. (log-log error vs. h/N)
5. **Transient convergence.** Tables/Plots verifying the expected second-order convergence of BE versus the analytical eigenvalue decomposition solution for a no-source core, demonstrating first-order solutions.
6. Tables of solution time and memory usage as functions of mesh resolutions, including evaluations of the feasibility of use of real-time operation at Minecraft's 20Hz update rate.

The following are desired deliverables and are to be implemented if time permits. They will be implemented in the future regardless to be published as a complete mod for the game.

1. Reactor temperature feedback curves to counteract runaway power increase when delayed supercritical.
2. Startup from no power using a source.
3. Demonstration of Java inputs and outputs to the game querying face couplings and control rod locations to update the solver at 20Hz.
4. Real-time visualization of the fast and thermal neutron flux distributions within the constructed core, including changes during control rod movement.
5. Implementation of reactor instrumentation displaying power, reactor period, and control rod positions through an in-game interface estimated with the inhour.

The final implementation will be accompanied by the source code, verification results, and representative reactor simulations. If time and computational performance permit, the solver will be integrated with Minecraft to demonstrate real-time reactor operation and power output.

References

[1] https://www.curseforge.com/minecraft/mc-mods/create

[2] https://www.curseforge.com/minecraft/mc-mods/create-crowns

[3] A. Y. Soliman, "Validation of COMSOL Multiphysics for PWR Power Distribution via 3D IAEA PWR Benchmark Problem," in *Proceedings of the 2017 COMSOL Conference*, Rotterdam, Netherlands, 2017.

[4] J. M. S. Mendes, S. B. Paixão, and S. de O. Vellozo, "New 3D Diffusion Code Based on the Nodal Polynomial Expansion," *Brazilian Journal of Radiation Sciences*, vol. 8, no. 3A, pp. 1–12, 2020.

[5] C. Fiorina, *Introduction to Nuclear Reactor Theory*, NUEN 301 Course Notes, Texas A&M University, Fall 2025, Sec. 9.12.5, pp. 228–232. Based on notes by M. Adams, with contributions by J. Ragusa, J. Till, and C. Fiorina.